

Day 26 — Capstone Part 2 — BLE GATT Environmental Service

Goal

Add a BLE GATT service to the capstone node that exposes temperature and humidity as notifiable characteristics, readable from any phone or BLE central.

What you will learn

- How to structure a multi-file Zephyr project with BLE in a separate module
- How to define two GATT characteristics in one service
- How to call `bt_gatt_notify` from application code after a sensor read
- How to verify both characteristics with nRF Connect

Overview

Building on Day 25 (BME280 + NVS), this day wires up the BLE stack so the node is visible and connectable. A custom 128-bit UUID service exposes temperature and humidity. Any connected central (phone, PC, gateway) can read the current value or subscribe to receive a notification each time the sensor is read.

Module structure

Split BLE into its own files to keep `main.c` clean:

```
src/  
├─ main.c           ← sensor + NVS logic, calls ble_sensor_init/  
notify  
├─ ble_sensor.h     ← public API  
└─ ble_sensor.c     ← GATT service definition
```

ble_sensor.h

```
#pragma once
#include <stdint.h>

void ble_sensor_init(void);
void ble_sensor_notify(int32_t temp_mdeg, int32_t hum_m);
```

ble_sensor.c — GATT service

```
#include <zephyr/bluetooth/bluetooth.h>
#include <zephyr/bluetooth/gatt.h>
#include "ble_sensor.h"

#define BT_UUID_SENSOR_SERVICE_VAL \
    BT_UUID_128_ENCODE(0xaabbccdd, 0x1234, 0x5678, 0xabcd, \
0x1234567890ab)
#define BT_UUID_TEMP_CHAR_VAL \
    BT_UUID_128_ENCODE(0xaabbccdd, 0x1234, 0x5678, 0xabcd, \
0x1234567890ac)
#define BT_UUID_HUM_CHAR_VAL \
    BT_UUID_128_ENCODE(0xaabbccdd, 0x1234, 0x5678, 0xabcd, \
0x1234567890ad)

#define BT_UUID_SENSOR_SERVICE
BT_UUID_DECLARE_128(BT_UUID_SENSOR_SERVICE_VAL)
#define BT_UUID_TEMP_CHAR
BT_UUID_DECLARE_128(BT_UUID_TEMP_CHAR_VAL)
#define BT_UUID_HUM_CHAR
BT_UUID_DECLARE_128(BT_UUID_HUM_CHAR_VAL)

static int32_t temp_val;
static int32_t hum_val;

static ssize_t read_temp(struct bt_conn *conn, const struct
bt_gatt_attr *attr,
                        void *buf, uint16_t len, uint16_t offset)
{
    return bt_gatt_attr_read(conn, attr, buf, len, offset,
                            &temp_val, sizeof(temp_val));
}

static ssize_t read_hum(struct bt_conn *conn, const struct
bt_gatt_attr *attr,
                        void *buf, uint16_t len, uint16_t offset)
{
    return bt_gatt_attr_read(conn, attr, buf, len, offset,
                            &hum_val, sizeof(hum_val));
}

BT_GATT_SERVICE_DEFINE(sensor_svc,
```

```

        BT_GATT_PRIMARY_SERVICE(BT_UUID_SENSOR_SERVICE),
        BT_GATT_CHARACTERISTIC(BT_UUID_TEMP_CHAR,
            BT_GATT_CHRC_READ | BT_GATT_CHRC_NOTIFY,
            BT_GATT_PERM_READ, read_temp, NULL, &temp_val),
        BT_GATT_CCC(NULL, BT_GATT_PERM_READ | BT_GATT_PERM_WRITE),
        BT_GATT_CHARACTERISTIC(BT_UUID_HUM_CHAR,
            BT_GATT_CHRC_READ | BT_GATT_CHRC_NOTIFY,
            BT_GATT_PERM_READ, read_hum, NULL, &hum_val),
        BT_GATT_CCC(NULL, BT_GATT_PERM_READ | BT_GATT_PERM_WRITE),
    );

    static const struct bt_data ad[] = {
        BT_DATA_BYTES(BT_DATA_FLAGS, BT_LE_AD_GENERAL |
BT_LE_AD_NO_BREDR),
        BT_DATA(BT_DATA_NAME_COMPLETE, CONFIG_BT_DEVICE_NAME,
            sizeof(CONFIG_BT_DEVICE_NAME) - 1),
    };

    void ble_sensor_init(void)
    {
        int ret = bt_enable(NULL);
        if (ret < 0) {
            LOG_ERR("bt_enable: %d", ret);
            return;
        }
        ret = bt_le_adv_start(BT_LE_ADV_CONN, ad, ARRAY_SIZE(ad),
NULL, 0);
        if (ret < 0) {
            LOG_ERR("adv_start: %d", ret);
        }
    }

    void ble_sensor_notify(int32_t temp_mdeg, int32_t hum_m)
    {
        temp_val = temp_mdeg;
        hum_val = hum_m;
        bt_gatt_notify(NULL, &sensor_svc.attrs[1], &temp_val,
sizeof(temp_val));
        bt_gatt_notify(NULL, &sensor_svc.attrs[3], &hum_val,
sizeof(hum_val));
    }

```

Additional prj.conf entries

```

CONFIG_BT=y
CONFIG_BT_PERIPHERAL=y
CONFIG_BT_DEVICE_NAME="SensorNode"
CONFIG_BT_GATT_DYNAMIC_DB=n

```

Integrating into main.c

After the NVS write in Day 25's `main()`, add:

```
ble_sensor_init();
ble_sensor_notify(last_temp_mdeg, last_hum_m);
/* Keep advertising for 2 seconds so a central can read */
k_sleep(K_SECONDS(2));
```

Why this matters

Separating the GATT service into its own module is a pattern used in every production BLE firmware. It makes the service testable in isolation, keeps `main.c` readable, and lets you swap the transport (BLE vs USB vs LoRa) without touching application logic.

Practice tasks

1. Open nRF Connect → scan → connect to “SensorNode”.
2. Subscribe to both characteristics — verify temperature and humidity notifications arrive.
3. Decode the raw bytes: temperature is `int32_t` milli-degrees C, humidity is `int32_t` milli-percent.
4. Add a third characteristic that exposes the boot count from NVS (Day 25).

Example folder

View the complete source code on GitHub: [src/capstone_wireless_sensor_node](#)

Next topic

Day 27 adds the final piece: a 60-second sleep cycle with System OFF to achieve sub- μ A quiescent current between readings.